

UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final Modelo

Disciplina: Qualidade de Software
Equipe: Ana Julia Chaves Souto da Costa - 569565, Gabriela Nogueira Coutinho - 556301
Projeto: Qutebrowser - https://github.com/qutebrowser/qutebrowser
Fork: https://github.com/gbrlcoutinho/qutebrowser
Pull Request: aberto — https://github.com/qutebrowser/qutebrowser/pull/9048
Commit SHA: 9f6e299812611d2d5f5b52e698b443898d90815f

1. Apresentação do projeto

O qutebrowser é um navegador web de código aberto, leve e focado em produtividade, desenvolvido em Python com a biblioteca Qt. O seu principal diferencial é permitir a navegação completa sem a necessidade do mouse, utilizando atalhos de teclado baseados no editor Vim. Ele é voltado para desenvolvedores, administradores de sistema e entusiastas de produtividade que buscam uma navegação rápida, minimalista e altamente customizável.

A suíte de testes fica na pasta tests/ e está organizada principalmente em testes unitários (tests/unit/) e testes end-to-end (tests/end2end/). O arquivo tests/conftest.py e os utilitários de tests/helpers/ dão suporte à configuração dos testes, fixtures e mocks.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Unknown Test	713
Conditional Test Logic	577
Undefined Test	130
Non-Functional Statement	78
Programming Paradigms Blend	72

Test smell	**Ocorrências detectadas**
Exception Handling	51
Redundant Print	46
Verbose Test	28
Sleepy Test	13
Total	1708

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	8.200
Testes com falha (Failed)	574
Erros (Errors)	592
Ignorados (Skipped)	871
Cobertura total (%)	67,97137684283128%

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	35.476
Complexidade ciclomática média das funções de teste	2,20
Índice de manutenibilidade médio dos arquivos de teste	61,56

3. Refatorações realizadas

3.1 Seleção das ocorrências

Para cumprir a meta de 40 ocorrências distribuídas igualmente em 4 tipos de test smells (10 ocorrências por tipo), foram selecionados os smells **Conditional Test Logic**, **Sleepy Test**,

Non-Functional Statement e **Redundant Print**. A escolha considerou a severidade indicada pela ferramenta CATSPY, equilibrando o ganho de qualidade da suíte com a segurança das alterações.

O Conditional Test Logic foi definido como prioridade por apresentar severidade alta no CATSPY. A presença de estruturas condicionais (*if*, *else*, *for*) dentro de métodos de teste esconde caminhos de execução e dificulta a identificação da causa exata de uma falha, sendo um problema crítico a ser corrigido via parametrização do Pytest ou divisão em testes independentes. O Sleepy Test foi incluído em seguida por possuir severidade média, visto que pausas arbitrárias (*time.sleep()*) geram testes instáveis (*flaky*) e aumentam desnecessariamente o tempo total de execução.

Para completar a cota exigida com refatorações objetivas e de baixo risco, foram escolhidos o Non-Functional Statement e o Redundant Print. ambos com severidade baixa no CATSPY. O primeiro foca na eliminação de instruções sem efeito operacional (como *pass*) ou na inclusão de asserções pendentes, enquanto o segundo limpa a saída do terminal ao remover comandos *print()* desnecessários.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Redundant Print	tests/test_conftest.py::test_installed_package	691604927	sim	sim	não
R02	Redundant Print	tests/unit/components/test_braveadblock.py::test_adblock_cache	ea5c7e5b6	sim	sim	não
R03	Redundant Print	tests/unit/completion/test_completionwidget.py::TestCompletionItemFocusPage::test_steps	509d3fca7	sim	sim	não
R04	Redundant Print	tests/unit/scripts/test_problemmatchers.py::test_patterns	7d8d52dad	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R05	Redundant Print	tests/unit/test_qutebrowser.py::TestLogFilter::test_invalid	0a1ae0a63	sim	sim	não
R06	Redundant Print	tests/unit/browser/test_pdfjs.py::test_generate_pdfjs_page	fe9e5d194	sim	sim	não
R07	Redundant Print	tests/unit/browser/test_pdfjs.py::test_generate_pdfjs_page_url	b8a42a763	sim	sim	não
R08	Redundant Print	tests/end2end/features/test_history_bdd.py::check_query	9f833c4f1	sim*	sim	não
R09	Redundant Print	tests/end2end/features/test_private_bdd.py::check_cookie	f3af60bc1	sim*	sim	não
R10	Redundant Print	tests/end2end/features/test_private_bdd.py::check_cookie_not_set	b4ef74b39	sim*	sim	não
R11	Non-Functional Statement	tests/unit/config/test_configfiles.py::TestYaml::test_yaml_config	8f744f840	sim	sim	sim — Conditional Test Logic
R12	Non-Functional Statement	tests/unit/misc/test_lineparser.py::TestBaseLineParser::test_double_open	f428c2616	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R13	Non-Functional Statement	tests/unit/misc/test_pakjoy.py::test_version_gate	3cd2c6ad0	sim	sim	não
R14	Non-Functional Statement	tests/unit/misc/test_sql.py::TestTransaction::test_failed_transaction	5f95de51a	sim	sim	não
R15	Non-Functional Statement	tests/unit/misc/test_sessions.py::TestSave::test_utf_8_invalid	e4a833f0a	sim	sim	não
R16	Non-Functional Statement	tests/unit/utils/test_qtutils.py::test_qdatastream_status_members	5c0422e71	sim	sim	sim — Conditional Test Logic
R17	Non-Functional Statement	tests/unit/utils/test_utils.py::test_parse_duration_hypothesis	d145769cb	sim	sim	não
R18	Non-Functional Statement	tests/helpers/test_helper_utils.py::test_nop_contextmanager	e27b7138c	sim	sim	não
R19	Non-Functional Statement	tests/unit/keyinput/test_keyutils.py::TestKeySequence::test_parse_hypothesis	d0db78107	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R20	Non-Functional Statement	tests/unit/misc/test_ipc.py::test_timeout	797bd5e36	sim	sim	não
R21	Sleepy Test	tests/unit/components/test_misccommands.py::test_debug_crash_segfault	d9b2a1cf2	sim	sim	não
R22	Sleepy Test	tests/unit/utils/test_debug.py::TestLogTime::test_duration	e1939c1f7	sim	sim	não
R23	Sleepy Test	tests/unit/commands/test_userscripts.py::test_killed_command	cb34d0984	sim	sim	não
R24	Sleepy Test	tests/unit/misc/test_guiprocess.py::test_double_start	418fac3ab	sim	sim	não
R25	Sleepy Test	tests/unit/misc/test_guiprocess.py::test_running	89083395a	sim	sim	não
R26	Sleepy Test	tests/end2end/fixtures/test_testprocess.py::TestWaitFor::test_other_text	ed9555369	sim	sim	não
R27	Sleepy Test	tests/end2end/fixtures/test_testprocess.py::TestWaitFor::test_existing_me	39c9663a2	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		ssage_already_waited				
R28	Sleepy Test	tests/end2end/fixtures/test_testprocess.py::TestWaitFor::test_existing_message	41e96d707	sim	sim	não
R29	Sleepy Test	tests/end2end/fixtures/test_testprocess.py::TestWaitFor::test_successful	91d1ebbe8	sim	sim	não
R30	Sleepy Test	tests/end2end/fixtures/test_testprocess.py::TestEnsureNotLogged::test_late_message	d107517d1	sim	sim	não
R31	Conditional Test Logic	tests/unit/browser/test_signalfilter.py::test_filtering	99ab91512	sim	sim	não
R32	Conditional Test Logic	tests/unit/browser/webkit/test_cookies.py::test_cookies_changed	a04f12d1e	sim	sim	não
R33	Conditional Test Logic	tests/unit/browser/test_history.py::TestAdd::test_add_url	fecc6c10b	sim	sim	não
R34	Conditional Test Logic	tests/unit/browser/test_navigate.py::TestIncDec::test_incdec	cf4b4bf7c	sim	não*	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R35	Conditional Test Logic	tests/unit/config/test_configcommands.py::TestSet::test_set_print	acad0b4e3	sim	sim	não
R36	Conditional Test Logic	tests/unit/config/test_configutils.py::TestFontFamilies::test_to_str	01cce2000	sim	sim	não
R37	Conditional Test Logic	tests/unit/browser/test_downloadview.py::test_get_menu_actions	850f5849b	sim	sim	não
R38	Conditional Test Logic	tests/unit/commands/test_argparser.py::test_type_conv_valid	0ef922026	sim	sim	não
R39	Conditional Test Logic	tests/unit/commands/test_argparser.py::test_type_conv_invalid (mensagem)	8af7b4487	sim	sim	não
R40	Conditional Test Logic	tests/unit/commands/test_argparser.py::test_type_conv_invalid (conversor)	913b3bedd	sim	sim	não

Nota de validação R01–R10: execução conjunta — 197 passed, 4 skipped, 4 xfailed e 2 erros de teardown nos testes end-to-end, associados ao aviso Qt “Populating font family aliases”. Na execução unitária sem E2E: 167 passed, 2 skipped, 1 xfailed. Os dois testes E2E afetados passaram quando reexecutados isoladamente filtrando somente esse aviso.

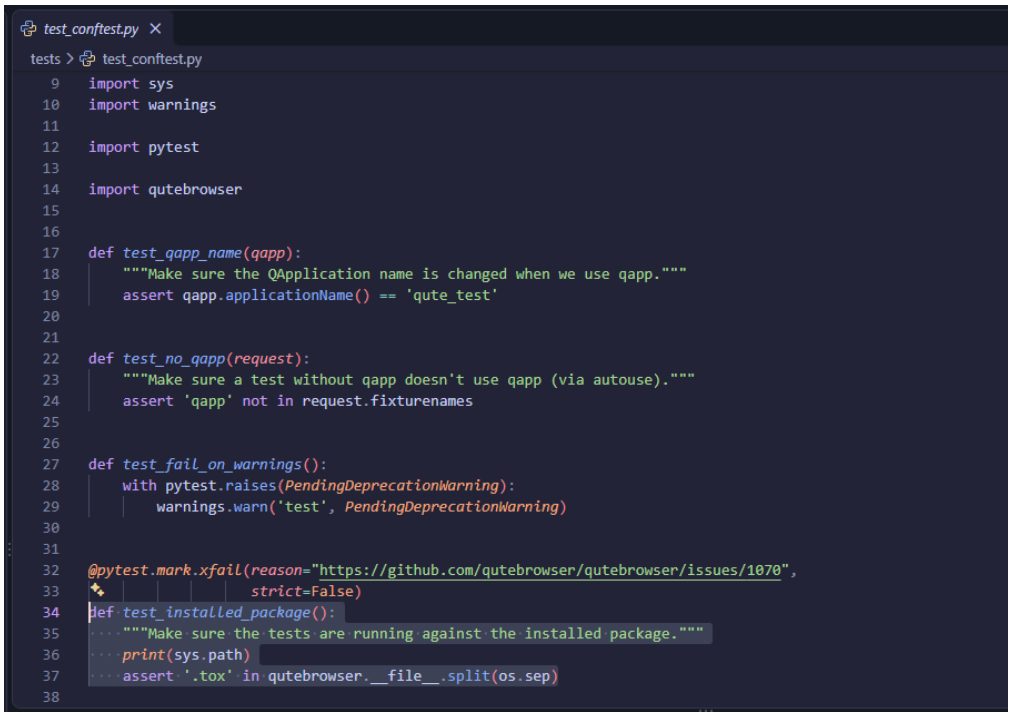
* R08–R10: consulte a nota de validação logo abaixo da tabela. Ela registra dois erros de teardown na execução conjunta de testes end-to-end e que os dois testes afetados passaram quando reexecutados isoladamente com o aviso do Qt filtrado; isso não significa que toda a execução conjunta passou.

* R34: o teste focal passou, mas o TEMPY continuou detectando Conditional Test Logic em test_incdec; por isso, o smell foi marcado como não removido.

3.3 Exemplos detalhados

R01. Redundant Print (tests/test_conftest.py::test_installed_package)

Antes:



Depois:



Por que era um test smell: Instruções print() em testes automatizados poluem a saída padrão (stdout) no terminal sem realizar verificações operacionais. Testes automatizados devem ser silenciosos e auto-avaliáveis através de asserções (assert), sem depender de inspeção visual humana no console.

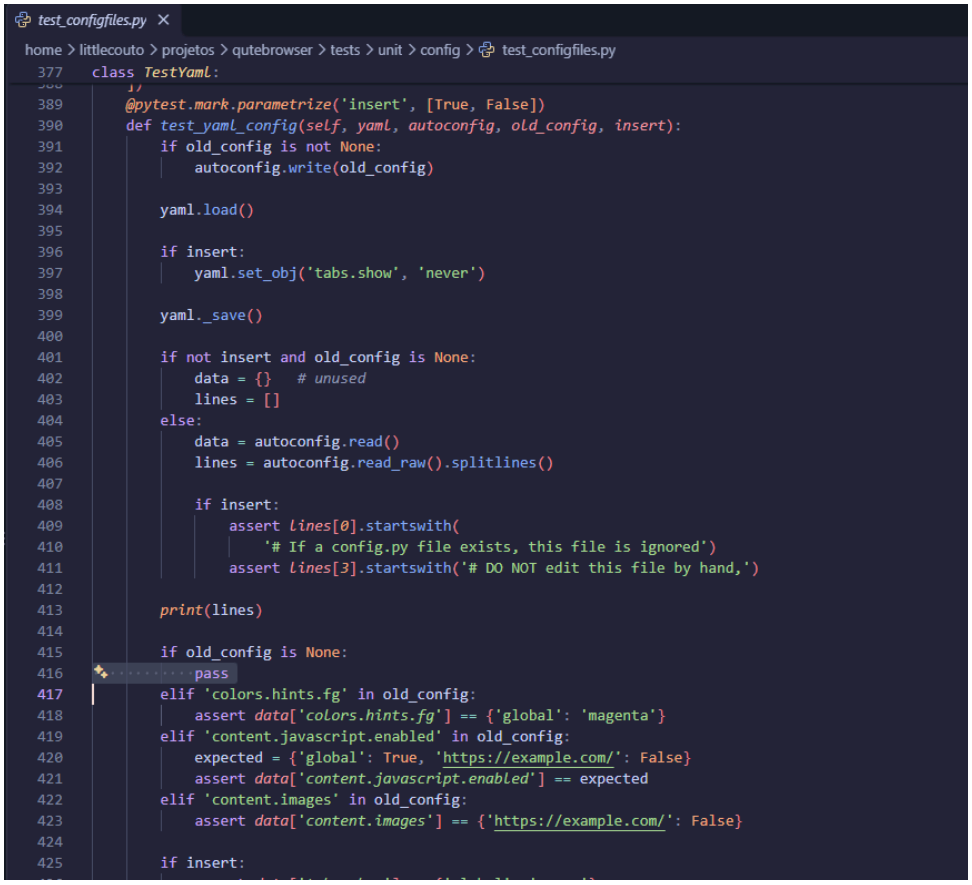
Estratégia do CATSPY aplicada: Remoção direta da chamada de exibição (print statement elimination). Adaptou-se a refatoração ao projeto removendo também a instrução import sys, que se tornou um unused import no arquivo após a exclusão do print().

Métricas do teste refatorado:

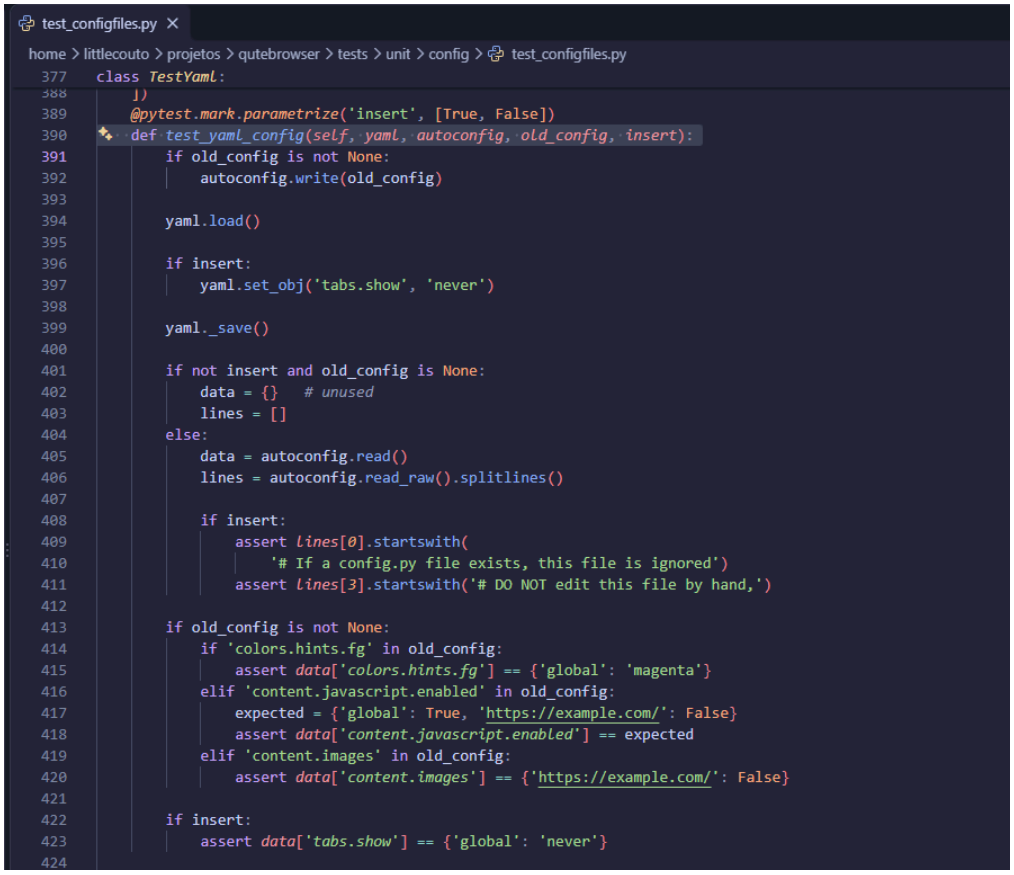
Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	17	16

R11. Non-Functional Statement (tests/unit/config/test_configfiles.py::TestYaml::test_yaml_config)

Antes:


test_configfiles.py
home > littlecouto > projetos > qutebrowser > tests > unit > config > test_configfiles.py
class TestYaml:
@pytest.mark.parametrize('insert', [True, False])
def test_yaml_config(self, yaml, autoconfig, old_config, insert):
if old_config is not None:
autoconfig.write(old_config)
yaml.load()
if insert:
yaml.set_obj('tabs.show', 'never')
yaml._save()
if not insert and old_config is None:
data = {} # unused
lines = []
else:
data = autoconfig.read()
lines = autoconfig.read_raw().splitlines()
if insert:
assert lines[0].startswith(
'# If a config.py file exists, this file is ignored')
assert lines[3].startswith('# DO NOT edit this file by hand,')
print(lines)
if old_config is None:
pass
elif 'colors.hints.fg' in old_config:
assert data['colors.hints.fg'] == {'global': 'magenta'}
elif 'content.javascript.enabled' in old_config:
expected = {'global': True, 'https://example.com/': False}
assert data['content.javascript.enabled'] == expected
elif 'content.images' in old_config:
assert data['content.images'] == {'https://example.com/': False}
if insert:

Depois:



Por que era um test smell: A instrução pass dentro do bloco if old_config is None: é uma instrução não-funcional (no-op). Trata-se de um desvio condicional explícito que não executa nenhuma ação nem realiza asserção, poluindo a estrutura lógica do teste.

Estratégia do CATSPY aplicada: Reestruturação da cadeia condicional encadeando as verificações sob a condição ativa if old_config is not None:. Isso elimina o ramo vazio com pass, tornando o fluxo do teste mais limpo e direto.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	17	17
SLOC do arquivo de teste (raw_por_arquivo_e_total)	1239	1237

R21. Sleepy Test (tests/unit/components/test_misccommands.py::test_debug_crash_segfault)

Antes:

```
32
33  @pytest.mark.skipif(utils.is_windows,
34                      reason="current CPython/win can't recover from SIGSEGV")
35  def test_debug_crash_segfault():
36      """Verify that debug_crash crashes as intended."""
37      caught = False
38
39      def _handler(num, frame):
40          """Temporary handler for segfault."""
41          nonlocal caught
42          caught = num == signal.SIGSEGV
43
44      with _trapped_segv(_handler):
45          # since we handle the segfault, execution will continue and run into
46          # the "Segfault failed (wat.)" Exception
47          with pytest.raises(Exception, match="Segfault failed"):
48              misccommands.debug_crash(typ='segfault')
49          time.sleep(0.001)
50      assert caught
51
```

Depois:

```
31
32  @pytest.mark.skipif(utils.is_windows,
33                      reason="current CPython/win can't recover from SIGSEGV")
34  def test_debug_crash_segfault():
35      """Verify that debug_crash crashes as intended."""
36      caught = False
37
38      def _handler(num, frame):
39          """Temporary handler for segfault."""
40          nonlocal caught
41          caught = num == signal.SIGSEGV
42
43      with _trapped_segv(_handler):
44          with pytest.raises(Exception, match="Segfault failed"):
45              misccommands.debug_crash(typ='segfault')
46      assert caught
47
```

Por que era um test smell: A espera fixa de 1 ms não sincronizava o teste com uma condição observável e tornava o tempo de parede parte desnecessária da execução. No caso, o manipulador de SIGSEGV atualiza o estado de forma síncrona antes de debug_crash lançar a exceção esperada; por isso, a pausa não acrescentava sincronização.

Estratégia do CATSPY aplicada: Remover o atraso fixo e basear a verificação no estado/evento observado. A asserção assert caught continua confirmando que o manipulador foi executado.

Métricas do teste refatorado: complexidade ciclomática da função — Antes: 2 | Depois: 2; SLOC do arquivo — Antes: 45 | Depois: 43. (Valores registrados no diário para R21.)

R31. Conditional Test Logic (tests/unit/browser/test_signalfilter.py::test_filtering)

Antes:

```
52
53  @pytest.mark.parametrize('index_of, emitted', [(0, True), (1, False)])
54  def test_filtering(objects, tabbed_browser_stubs, index_of, emitted):
55      browser = tabbed_browser_stubs[0]
56      browser.widget.current_index = 0
57      browser.widget.index_of = index_of
58      objects.signaller.signal.emit('foo')
59      if emitted:
60          assert objects.signaller.filtered_signal_arg == 'foo'
61      else:
62          assert objects.signaller.filtered_signal_arg is None
63
```

Depois:

```
52
53  @pytest.mark.parametrize('index_of, expected', [(0, 'foo'), (1, None)])
54  def test_filtering(objects, tabbed_browser_stubs, index_of, expected):
55      browser = tabbed_browser_stubs[0]
56      browser.widget.current_index = 0
57      browser.widget.index_of = index_of
58      objects.signaller.signal.emit('foo')
59      assert objects.signaller.filtered_signal_arg == expected
60
```

Por que era um test smell: O teste escolhia em tempo de execução qual asserção executar com base em emitted. Isso reunia dois resultados esperados em um fluxo condicional, tornando os cenários menos explícitos.

Estratégia do CATSPY aplicada: Representar o resultado esperado diretamente nos casos parametrizados e usar uma asserção única (filtered_signal_arg == expected), sem condicional dentro do teste.

Métricas do teste refatorado: complexidade ciclomática da função — Antes: 4 | Depois: 2; SLOC do arquivo — Antes: 64 | Depois: 61. (Valores registrados no diário para R31.)

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	1.708	1.662	−46
Testes aprovados	8.200	8.199	−1
Testes com falha	574	574	0

Métrica	Antes	Depois	Variação
Cobertura total (%)	67,97%	68,55%	+0,58 p.p.
SLOC dos testes	35.476	35.465	−11
Complexidade ciclomática média das funções de teste	2,2001	2,1940	−0,0060
Índice de manutenibilidade médio dos arquivos de teste	61,5554	61,6228	+0,0675

Test smell	Antes	Depois	Variação
Redundant Print	46	35	−11
Non-Functional Statement	78	68	−10
Sleepy Test	13	3	−10
Conditional Test Logic	577	569	−8
Total	1.708	1.662	−46

5. Análise dos resultados

Com base nas tabelas das Seções 2, 3 e 4, responda:

Quais métricas melhoraram e quais pioraram?

O total de ocorrências detectadas pelo TEMPY caiu de 1.708 para 1.662 (−46). A cobertura subiu de 67,97% para 68,55% (+0,58 ponto percentual); o SLOC dos testes caiu de 35.476 para 35.465 (−11); a complexidade ciclomática média caiu de 2,2001 para 2,1940 (−0,0060); e o índice médio de manutenibilidade subiu de 61,5554 para 61,6228 (+0,0675). O número de testes aprovados caiu de 8.200 para 8.199, as falhas permaneceram em 574 e os erros caíram de 592 para 591. As diferenças são descritivas e, isoladamente, não provam causalidade.

Alguma refatoração introduziu novos test smells? Quais?

Na comparação dos CSVs TEMPY antes/depois por arquivo, tipo e método, duas novas detecções de Conditional Test Logic apareceram entre R01–R20: R11 (test_yaml_config, contagem no método de 6 para 7) e R16 (test_qdatastream_status_members, de 0 para 1). Ambas são smells secundários associados às alterações; a atividade não exige refatorar outros tipos que surjam como consequência. Nas outras 18 ocorrências não houve aumento de contagem de smells no respectivo método. No agregado, nenhuma categoria aumentou e o total caiu de 1.708 para 1.662 (−46), mas esse total sozinho não revela os efeitos por ocorrência.

O resultado dos testes ou a cobertura mudou?

Sim. Antes/depois: 8.200/8.199 aprovados, 574/574 falhas, 592/591 erros, 871/871 ignorados; houve também 58 xfails e 32 reruns em ambas as execuções. A cobertura total passou de

67,97% para 68,55% (+0,58 p.p.). As execuções inicial e final não foram totalmente verdes; os resultados parciais e limitações do ambiente devem permanecer explícitos.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Pela natureza das alterações registradas, Redundant Print tendeu a ser mais direto quando a saída era apenas diagnóstica. Non-Functional Statement exigiu avaliar se cada instrução aparentemente sem efeito era realmente dispensável. Sleepy Test e Conditional Test Logic exigiram mais cuidado para preservar sincronização, caminhos e expectativas; no qutebrowser, sinais, event loop e processos auxiliares tornaram inadequada uma aplicação mecânica dos exemplos do catálogo.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Consultamos a página individual do CATSPY para cada tipo e registramos o link e a estratégia no diário de bordo. Usamos a definição e os efeitos negativos para avaliar cada candidato, e os exemplos/estratégias como orientação; não tratamos a detecção automática como prova suficiente de que uma instrução podia ser removida.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

As definições, os efeitos negativos e, principalmente, os exemplos de refatoração deram um ponto de partida. Para Sleepy Test, o princípio útil foi substituir atrasos arbitrários por sincronização determinística; para Conditional Test Logic, explicitar cenários e expectativas com parametrização. A aplicação exigiu adaptação aos sinais/event loop do Qt e aos processos auxiliares do qutebrowser.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim. As estratégias precisaram ser adaptadas à sincronização por sinais/event loop do Qt e aos processos auxiliares do qutebrowser; nos testes Sleepy, a meta foi substituir atrasos arbitrários por espera determinística por evento/estado ou tempo controlado, não apenas trocar uma pausa por outra. Em R34, a refatoração não eliminou toda a detecção do TEMPY no método test_incdec, e isso foi registrado como resultado parcial, sem apresentar a ocorrência como removida.

O que faltou ou poderia ser melhorado no catálogo?

Uma tabela oficial de correspondência entre os nomes de saída do TEMPY e os itens do CATSPY ainda não foi confirmada. Sua publicação e a clareza sobre quais ocorrências são elegíveis ajudariam a evitar equivalências inferidas apenas pelo filtro do site.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Gabriela Coutinho	R01 a R20	20
Ana Julia Chaves	R21 a R40	20

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

A principal dificuldade foi obter uma execução de testes reproduzível no macOS/arm64: a linha de base tinha falhas e erros, houve conflito de avisos de cobertura e alguns testes end-to-end tiveram erros de teardown associados a um aviso do Qt. Os testes afetados foram reexecutados isoladamente com o aviso filtrado e passaram; essa limitação foi registrada, não tratada como sucesso da execução conjunta. Também foi necessário avaliar caso a caso se uma espera ou instrução aparentemente inerte fazia parte da sincronização/semântica do teste. As quatro branches de smells foram integradas à main do fork e o PR foi aberto no repositório original.

O que você aprendeu com a atividade?

Aprendi que o detector aponta candidatos, mas cada ocorrência precisa ser interpretada no contexto para preservar o comportamento do teste. Nas ocorrências trabalhadas, removi esperas de parede substituindo-as por sincronização por estado/evento ou tempo controlado; nos testes com lógica condicional, deixei os cenários explícitos por parametrização e resultados esperados. A validação focal, a conferência TEMPY e um commit por ocorrência tornam a mudança rastreável, enquanto os resultados parciais precisam ser relatados sem mascarar falhas.

9. Contribuição (Pull Request)

O PR foi aberto no repositório original com as refatorações das quatro categorias: https://github.com/qutebrowser/qutebrowser/pull/9048. Resposta dos mantenedores: ainda não registrada.

10. Observações gerais

A linha de base do Pytest não foi verde: houve testes aprovados, falhas, erros e itens ignorados. Para coletar resultados válidos neste Mac, a execução acadêmica usou TMPDIR=/tmp e filtros temporários para avisos de cobertura herdados pelos processos filhos; os mesmos parâmetros foram repetidos na medição final. As medições “before” e “after” estão preservadas em metrics-before-* e metrics-after-*; a comparação está preenchida na Seção 4. A suíte continuou com falhas e erros após as refatorações. As quatro branches foram integradas à main do fork e enviadas ao fork; o PR correspondente está identificado na Seção 9.